Python Interpreter Guide

A simple, text-only reference for learning and using the Python interpreter.

1. What Is the Python Interpreter?

The Python interpreter is the program that reads Python source code and executes it. It translates your instructions into actions performed by the computer.

When you write a Python file, the interpreter reads the file, checks its syntax, prepares the code for execution, and runs it. Python is commonly described as an interpreted language because code is executed by an interpreter rather than being compiled directly into a standalone machine-code executable.

The interpreter can be used in two main ways: interactively, where you type one command at a time, or as a script runner, where you execute an entire Python file.

2. Checking Whether Python Is Installed

Open a terminal.

On Windows, open Command Prompt or PowerShell.
On macOS or Linux, open Terminal.

Type:

python --version

If that does not work, try:

python3 --version

A successful command displays something similar to:

Python 3.12.4

If Python is not installed, download it from the official Python website. During installation on Windows, select the option to add Python to the system PATH.

3. Starting the Interactive Interpreter

To start the interactive interpreter, open a terminal and type:

python

or:

python3

You will see a prompt similar to:

```
>>>
```

The three greater-than symbols are the Python prompt. They mean Python is waiting for you to enter a command.

Example:

```
>>> 2 + 3
5

>>> print("Hello, Python")
Hello, Python
```

The interpreter evaluates each expression immediately and displays the result.

4. Exiting the Interactive Interpreter

To exit the interactive interpreter, type:

```
exit()
```

or:

```
quit()
```

You can also press Ctrl+D on macOS or Linux. On Windows, press Ctrl+Z followed by Enter in Command Prompt.

After exiting, you return to the normal operating-system terminal prompt.

5. Running a Python Script

Create a file named hello.py using any text editor.

Add this line:

```
print("Hello, Python")
```

Save the file. In the terminal, move to the folder containing the file and run:

```
python hello.py
```

or:

```
python3 hello.py
```

The interpreter reads the entire file and executes it from top to bottom.

Example output:

Hello, Python

6. The Python Prompt and REPL

REPL means Read, Evaluate, Print, Loop.

Read: Python reads the line you typed.
Evaluate: Python calculates or executes the expression.
Print: Python displays the result, if there is one.
Loop: Python waits for the next command.

The REPL is useful for testing small pieces of code, checking function behavior, exploring libraries, and learning syntax.

Example:

```
>>> 10 / 4
2.5

>>> 10 // 4
2

>>> 10 % 4
2

>>> 2 ** 3
8
```

7. Variables and Values

A variable is a name that refers to a value.

```
>>> name = "Aravind"
>>> age = 43
>>> height = 5.9
>>> is_manager = True
```

Python determines the type automatically.

```
>>> type(name)
<class 'str'>
>>> type(age)
<class 'int'>
>>> type(height)
<class 'float'>
>>> type(is_manager)
<class 'bool'>
```

Variable names should be descriptive. Use lowercase words separated by underscores, such

as total_amount, user_name, or model_accuracy.

8. Basic Data Types

Python has several common built-in data types.

Integer: whole numbers, such as 10, -5, or 1000.
Float: decimal numbers, such as 3.14 or -0.5.
String: text enclosed in single or double quotes, such as "Python".
Boolean: True or False.
None: represents the absence of a value.

Examples:

```
>>> count = 25
>>> price = 99.99
>>> language = "Python"
>>> active = True
>>> result = None
```

9. Strings

Strings store text.

```
>>> greeting = "Hello"
>>> name = "Aravind"
>>> message = greeting + ", " + name
>>> message
'Hello, Aravind'
```

You can repeat a string:

```
>>> "-" * 10
'----------'
```

You can find its length:

```
>>> len(message)
14
```

You can convert case:

```
>>> message.upper()
'HELLO, ARAVIND'

>>> message.lower()
'hello, aravind'
```

10. Lists

A list stores an ordered collection of items.

```
>>> skills = ["Python", "SQL", "Machine Learning"]
>>> skills[0]
'Python'
>>> skills[-1]
'Machine Learning'
```

Lists are mutable, meaning their contents can change.

```
>>> skills.append("LLMs")
>>> skills
['Python', 'SQL', 'Machine Learning', 'LLMs']

>>> skills[1] = "Pandas"
>>> skills
['Python', 'Pandas', 'Machine Learning', 'LLMs']
```

11. Tuples

A tuple is similar to a list, but it cannot be changed after creation.

```
>>> location = ("Chennai", "India")
>>> location[0]
'Chennai'
```

Trying to modify a tuple causes an error:

```
>>> location[0] = "Mumbai"
TypeError: 'tuple' object does not support item assignment
```

Use tuples for fixed groups of related values, such as coordinates or configuration values.

12. Dictionaries

A dictionary stores key-value pairs.

```
>>> person = {"name": "Aravind", "role": "Technology Manager", "experience": 20}
>>> person["name"]
'Aravind'
```

You can add or update entries:

```
>>> person["city"] = "Chennai"
>>> person["experience"] = 21
```

You can list keys and values:

```
>>> person.keys()
dict_keys(['name', 'role', 'experience', 'city'])

>>> person.values()
dict_values(['Aravind', 'Technology Manager', 21, 'Chennai'])
```

13. Operators

Python supports arithmetic, comparison, logical, and assignment operators.

Arithmetic operators:
+ addition
- subtraction
* multiplication
/ division
// floor division
% remainder
** exponentiation

Comparison operators:
== equal to
!= not equal to
> greater than
< less than
>= greater than or equal to
<= less than or equal to

Logical operators:
and, or, not

Examples:

```
>>> 7 > 3
True
>>> 7 == 7
True
>>> True and False
False
>>> not True
False
```

14. Conditional Statements

Use if, elif, and else to make decisions.

```
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Good")
```

```
else:
    print("Needs improvement")
```

Indentation is important in Python. Four spaces are commonly used for each indentation level.

Example in the interpreter:

```
>>> score = 85
>>> if score >= 75:
...     print("Good")
...
Good
```

15. Loops

Use a for loop to repeat over a sequence.

```
>>> for language in ["Python", "SQL", "ML"]:
...     print(language)
...
Python
SQL
ML
```

Use range() to repeat a fixed number of times.

```
>>> for i in range(3):
...     print(i)
...
0
1
2
```

Use a while loop when repetition depends on a condition.

```
>>> count = 0
>>> while count < 3:
...     print(count)
...     count += 1
...
0
1
2
```

16. Functions

A function is a reusable block of code.

```
>>> def greet(name):
...     return "Hello, " + name
...
>>> greet("Aravind")
'Hello, Aravind'
```

Functions can accept multiple arguments.

```
>>> def add(a, b):
...     return a + b
...
>>> add(3, 4)
7
```

Functions improve readability and reduce repeated code.

17. Modules and Imports

A module is a Python file containing reusable code. Use import to access it.

```
>>> import math
>>> math.sqrt(16)
4.0
>>> math.pi
3.141592653589793
```

You can import a specific function:

```
>>> from math import sqrt
>>> sqrt(25)
5.0
```

You can give a module a shorter name:

```
>>> import numpy as np
```

18. Standard Library Examples

Python includes many built-in modules.

os: interacts with the operating system.
sys: provides access to interpreter settings and command-line arguments.
json: reads and writes JSON data.
datetime: works with dates and times.
random: generates random values.
csv: reads and writes CSV files.

Example:

```
>>> import random
>>> random.randint(1, 10)
7

>>> from datetime import date
>>> date.today()
datetime.date(2026, 10, 11)
```

19. Handling Errors

Errors are called exceptions. Use try and except to handle them safely.

```
>>> try:
...     number = int("abc")
... except ValueError:
...     print("Please enter a valid number")
...
Please enter a valid number
```

Common exceptions include:

ValueError: invalid value.
TypeError: wrong data type.
IndexError: list index does not exist.
KeyError: dictionary key does not exist.
FileNotFoundError: file cannot be found.
ZeroDivisionError: division by zero.

20. Reading Command-Line Arguments

Python scripts can receive values from the terminal.

Create args.py:

```
import sys
print("Script name:", sys.argv[0])
print("Arguments:", sys.argv[1:])
```

Run:

```
python args.py alpha beta
```

Output:

```
Script name: args.py
Arguments: ['alpha', 'beta']
```

21. Working with Files

To read a file:

```
with open("notes.txt", "r") as file:
    content = file.read()
    print(content)
```

To write a file:

```
with open("output.txt", "w") as file:
    file.write("Python interpreter guide")
```

To append text:

```
with open("output.txt", "a") as file:
    file.write("\nNew line")
```

Using with automatically closes the file after the block finishes.

22. Virtual Environments

A virtual environment is an isolated workspace for a Python project. It prevents packages from one project from conflicting with another.

Create one:

```
python -m venv .venv
```

Activate it:

On Windows:
```
.venv\Scripts\activate
```

On macOS or Linux:
```
source .venv/bin/activate
```

After activation, your terminal prompt usually shows (.venv).

Install packages:

```
pip install pandas
```

Save installed packages:

```
pip freeze > requirements.txt
```

Install from a requirements file:

```
pip install -r requirements.txt
```

23. Using pip

pip is Python's package installer.

Check pip:

pip --version

Install a package:

pip install requests

Upgrade a package:

pip install --upgrade requests

Remove a package:

pip uninstall requests

List installed packages:

pip list

Always use a virtual environment when installing third-party packages.

24. Useful Interpreter Commands

help(): opens interactive help.
help(print): shows documentation for print.
dir(str): lists string methods.
type(value): shows the type of a value.
id(value): shows the identity of an object.

Examples:

```
>>> help(len)
>>> dir(list)
>>> type(3.14)
<class 'float'>
```

25. Debugging Basics

When an error occurs, read the traceback from bottom to top. The last line usually explains the error.

Example:

```
Traceback (most recent call last):
  File "example.py", line 2, in <module>
```

```
    print(10 / 0)
ZeroDivisionError: division by zero
```

To debug, check:

Variable names and spelling.
Data types.
Indentation.
Missing colons.
Incorrect file paths.
Index positions.

You can print values during execution:

```
print("Debug:", variable_name)
```

26. Python Interpreter Modes

Python can run in several modes.

Interactive mode: type commands one at a time.
Script mode: run a .py file.
Module mode: run a file as a module using python -m module_name.
Command mode: execute a short command using python -c "command".

Example:

```
python -c "print(2 + 2)"
```

Output:

```
4
```

27. Python Execution Model

When Python runs a script, it performs several steps.

1. It reads the source file.
2. It checks the syntax.
3. It compiles the code into bytecode.
4. The Python virtual machine executes the bytecode.
5. It produces output or changes program state.

This process is mostly automatic. As a learner, the important point is that Python code is executed line by line in the order it appears, subject to function definitions, loops, and control flow.

28. Naming Conventions

Use clear and consistent names.

Variables: total_amount, user_name.
Functions: calculate_total(), load_data().
Classes: DataProcessor, ModelTrainer.
Constants: MAX_RETRIES, DEFAULT_TIMEOUT.

Avoid single-letter names except for simple loop counters. Avoid names that conflict with built-in functions, such as list, dict, str, or type.

29. Comments and Documentation

Comments explain code to humans. Python ignores them during execution.

```
# This is a single-line comment

"""
This is a multiline string often used as a docstring.
"""

def calculate_area(width, height):
    """Return the area of a rectangle."""
    return width * height
```

Good comments explain why code exists, not merely what it does.

30. Best Practices

Use Python 3, not Python 2.
Use a virtual environment for every project.
Write small functions with clear purposes.
Use meaningful variable names.
Handle expected errors with try and except.
Test small pieces of code in the REPL.
Keep scripts organized in folders.
Use version control such as Git.
Read error messages carefully.
Install only necessary packages.

31. Common Beginner Mistakes

Forgetting a colon after if, for, while, or def.
Using inconsistent indentation.
Confusing = with ==.
Calling a function before defining it.
Modifying a list while looping through it.
Using an undefined variable.
Forgetting quotes around strings.

Using the wrong Python command, such as python instead of python3.
Installing packages globally instead of in a virtual environment.

32. Practice Exercises

Exercise 1: Start the Python interpreter and calculate 25 multiplied by 4.

Exercise 2: Create a variable called city with the value Chennai and print it.

Exercise 3: Create a list of three programming languages and print the second item.

Exercise 4: Write a function that accepts a number and returns its square.

Exercise 5: Write a script that prints numbers from 1 to 10.

Exercise 6: Create a dictionary with your name, role, and city, then print each value.

Exercise 7: Write a script that asks for two numbers and prints their sum.

Exercise 8: Create a virtual environment and install the requests package.

33. Sample Complete Script

Save this as calculator.py:

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def main():
    print("Simple Calculator")
    a = float(input("Enter first number: "))
    b = float(input("Enter second number: "))
    print("Sum:", add(a, b))
    print("Difference:", subtract(a, b))

if __name__ == "__main__":
    main()
```

Run it:

```
python calculator.py
```

34. Quick Reference

Start interpreter: python or python3
Exit interpreter: exit()
Run script: python script.py
Check version: python --version
Create virtual environment: python -m venv .venv
Activate on Windows: .venv\Scripts\activate
Activate on macOS/Linux: source .venv/bin/activate
Install package: pip install package_name
List packages: pip list
Show help: help(object)
Show type: type(object)
List attributes: dir(object)

35. Summary

The Python interpreter is the core tool that executes Python code. You can use it interactively for quick experiments or run complete scripts from files.

Learn the REPL first. Practice variables, data types, operators, loops, functions, imports, files, and error handling. Then use virtual environments and pip to manage real projects.

With consistent practice, the Python interpreter becomes a powerful environment for automation, data analysis, machine learning, and AI development.